UNIVERSIDADE KIMPA VITA
INSTITUTO POLITÉCNICO DO UÍGE
CURSO DE LICENCIATURA EM ENGENHARIA INFORMÁTICA
SISTEMA DE GESTÃO DE BASE DE DADOS II

2026

TEMA: Design e Implementação de uma Base de Dados para um sistema de Gestão Clínica Médicas

RELATÓRIO TÉCNICO

POR:

FRANCISCO BÁRULIO FIGUEIREDO ANTÓNIO, JOÃO JOSÉ ANTÓNIO DA SILVA e KUMBI MANUEL DOMINGOS

Startup: TAKETWARE

Tecnológias: brModelo, MySQL Workbanch e XAMPP (phpmyadmin).

ORIENTADOR

Moyo Kanivengidio, MSc.

Sumário

INTRODUÇÃO 2
ENQUADRAMENTO TEÓRICO 3
ANÁLISE DOS REQUISITOS 4
1. Identificação de Entidades e Atributos 4
2. Cardinalidade e Relacionamentos 5
3. Análise crítica de Restrições e Segurança 5
MODELAGEM 7
A. Modelo conceptual (MER) 7
Diagrama Entidade_Relacionamento (DER) 7
#. Justificação das Cardinalidades 7
#. Classificação das Entidades 8
A. Modelo Lógico 9
1. Diagrama de Estrutra Lógica 9
2. Análise da Normalização 9
3. Dicionário de Relações 10
B. IMPLEMENTAÇÃO FÍSICA (DDL) 10
1. Estrutura de Domínios e Fundação 10
#. Criação da Base de Dados 10
DML E POPULAÇÃO DE DADOS 13
#. Povoamento de cada tabela 13
CONSULTAS DE DADOS 17
#. Tipos de Consultas de dados 17
a. Pacientes por Especialidade no Último Mês 17
b. Médicos com mais consultas na próxima Semana 18
c. Histórico Médico completo de um paciente 19
d. Receita total por especialidade Médica 19
e. Pacientes com alergias a medicamentos específicos 20
f. Relatório de faturas pendentes 20
ANÁLISE CRÍTICA 21
Virtudes do sistema 21
Dificuldade encotradas 21
CONCLUSÃO 22

INTRODUÇÃO

Com o avanço das tecnogias de informação e comunicação, a quantidade de dados produzidos e armazenados pelas organizações tem aumentado de forma significativa. Nesse contexto, os **Bancos de Dados** tornaram-se componentes essencias para a gestão, organização e segurança da informação.

Os **bancos de dados** permitem armazenar grandes volumes de dados de forma estruturada, garantindo a integridade, consistência, segurança e facilidade de acesso à informação. Para tal, utilizam-se os **Sistemas de Gerenciamento de Banco de Dados (SGBD)**, que possibilitam a criação, manuntenção e controlo dos dados armazenados.

Deste modo, o presente trabalho, com o **tema: Design e Implementação de uma Base de Dados para um sistema de Gestão Clínica Médicas**, tem como objectivo geral analisar os conceitos básicos de base de dados e a sua modelagem, destacando a importância do **Modelo Entidade-Relacionamento** no desenvolvimento de sistemas de informação. Como objectivo específicos, pretende-se compreender o funcionamento dos SGDBs, e compreender as etapas do projeto de uma base de dados.

O sistema foi desenhado para gerir solicitações dos pacientes, triagem dos pedidos (ou serviços) e a designação de técnicos responsáveis para intervenções presenciais, com foco na complexidade dos dados de saúde, segurança e integridade.

OBJECTIVO: Projectar, normalizar e implementar uma base de dados relacional para um sistema de gestão de clínicas médicas, com foco na complexidade dos dados de saúde, segurança e integridade, demonstrar a aplicação de todas as fases do design da base de dados.

ENQUADRAMENTO TEÓRICO

A base de dados pode ser definido como um conjunto de dados relacionados, organizados de forma a representar irnformações relevantes para uma determinada organização. Estes dados são armazenados de maneira estruturada, permitindo o seu acesso, actualização e gestão de forma eficiente.

Para a gestão de uma base de dados utiliza-se o Sistema de de Gerenciamento de **Banco de Dados (SGDB),** que é um software responsável por controlar o acesso aos dados, garantir a segurança da inrformação, reduzir a redundância e assegurar a integridade dos dados. Entre as principais funções do SGDB destacam-se o controlo de acesso, a realização de cópias de segurança (**backup**), recuperação de dados e suporte a multíplos utilizadores.

No desenvolvimento de uma base de daodso, a modelagem de dados assume um papel fundamental, pois permite representar de forma abstrata a realidade dos sistema a ser implementado. Um dos modelos mais utilizados é o **Modelo Entidade-Relacionamento (ER)**, que descreve os dados por meio de entidades, atributos e relacionamentos, facilitando a compreensão do sistema tanto por utilizadores quanto por desenvolvedores.

Além disso, as bases de dados apresentam diferentes níveis de abstração, nomeadamente o nível físco, o nível conceptual e o nível de visualização. Estes níveis permitem separar os detalhes internos de armazenamento da forma como os dados são apresentados aos utilizadores finais. Assim, o enquadramento teórico evidencia que as base de dados e os SGDBs são ferramentas indispensáveis para a organização, gestão e utilização da informação nas instituições modernas

ANÁLISE DOS REQUISITOS

Nesta fase do projecto, temos por objectivo obedecer, cumprir com os requisitos estabelecidos no enunciado do trabalho prático, bem como criar e desenvolver modelos que descrevem estáticamente o que o projecto em si deve fazer, e não deve ser feito, tudo de acordo ao enucniado do trabalho.

No entanto, partindo para a identificação das necessidades informais da Clínica médica, definindo a estrutura lógica que garentirá a persistência, integridade e segurança dos dados clínicos e administrativos.

1. Identificação de Entidades e Atributos

Uma entidade é uma representação de um conjunto de inforamções sobre determinado conceito do sistema. Toda entidade possui Atributos, que são as informações que referenciam a entidade.

Abaixo, temos de maneira detalhada as entidades, e os seus respectivos atributos, refenciando tipos de dados e restrições de integridade.

1.1- Paciente

- Id_paciente: INT (Primary key, auto-incremento);
- Nome: Varchar (100) (Not null);
- Bi: Varchar (14) (Unique, Not null);
- Data_nascimento: Date (nut null);
- Contacto: Varchar (9);
- Historico_alergias: Text (Aqui nós grupo, premitimos registos nulos se eventualmente o paciente não tiver alergias);

1.2- Medico

- Id_paciente: Int (primary key, auto_increment);
- Nome: Varchar (100) (Not null);
- Carteira_profissional: varchar (20) (unique, not null);

1.3- Especialidade

- Id_especialidade: Int (primary key, auto-increment);
- Nome_especialidade: varchar (50) (unique, not null);

1.4- Consulta

- Id_consulta: Int (primary key, auto-increment);
- Data_consulta: Date (not null);
- Hora_consulta: time (not null);
- Estado: enum (‘agenda’, ‘realizada’, ‘cancelada’, ‘reagendada’) (default ‘agendada’);
- Id_paciente: int (foreign key, not null);
- Id_medico: int (foreign key, not null);

1.5- HistoricoMedico

- Id_historico: int (primary key, auto-increment);
- Id_paciente: int (foreign key, not null);
- Id_consulta: int (foreign key, not null);

- Diagnostico: Text (not null);
- Tratamento_prescrito: Text;

1.6- Exame

- Id_exame: int (primary key, auto-increment);
- Tipo_exame: varchar (100) (not null);
- Data_exame: date (not null);
- Resultado: text
- Id_paciente: int (foreign key, not null);
- Id_paciente: int (foreign key, not null);
- Id_consulta: int (foreign key);

1.7- Fatura

- Id_fatura: int (primary key, auto-increment);
- Valor_total: decimal (10,2) (not null, check > 0);
- Data_emissao: datatime (default corrent_timestamp);
- Estado_pagamento: Enum ('Pendente', 'Pago', 'Cancelado') (default 'Pendente');
- Id_consulta: int (foreign key, unique);

1.8- Medicamento

- Id_medicamento: int (primary key, auto-increment);
- Nome_genrico: varchar (100) (not null);
- Dosagem_padrao: varchar (50);

2. Cardinalidade e Relacionamentos

Nesta fase, descrevemos as associações entre entidades, fundamentais para a integridade referencial do sistema:

- **Paciente – Consulta (1:N):** Um paciente pode realizar múltiplas consultas ao longo do tempo, mas cada consulta pertence à apenas um paciente.
- **Medico – Consulta (1:N):** Um médico possui uma agenda com várias consultas, mas cada registo de consulta é atribuído à um único médico.
- **Especialidade – Medico (1:N):** Uma especialidade (ex: Cardiologia) pode ter vários médico, mas cada médico está vinculado a uma especialidade principal (neste modelo).
- **Paciente – HistoricoMedico (1:N**): Um paciente acumula vários registos no seu histórico ao longo das suas visitas.
- **Consulta – Fatura (1:1):** Para cada consulta realizada, gera-se uma fatura única.
- **Paciente – Exame (1:N):** Um paciente pode realizar diversos exames.

3. Análise crítica de Restrições e Segurança

Para além das chaves primárias e estrangeiras, usamos também o seguinte:

- **Integridade Temporal:** Uso de **CHECK** para que a data_consulta não seja anterior a data de registo (evitando anomalias cronológicas).

- **Consistência Financeira**: Aplicação de **CHECK (valor_total > 0)** na entidade Fatura, impedindo lançamentos contabilisticos erróneos.
- **Segurança de dados:** Atributos como diagnostico e resultado de exames foram isolados em tabelas específicas para permitir que no futuro, níveis de acesso **(Privilégios SQL)** sejam aplicados de forma a que apenas pessoal médico autrizado visualize tais dados.
- **Unicidade:** O uso de **UINIQUE** em campos como BI e Carteira Profissional impede a duplicidade e registos de cidadãos e profisssionais no sistema.

MODELAGEM

A. Modelo conceptual (MER)

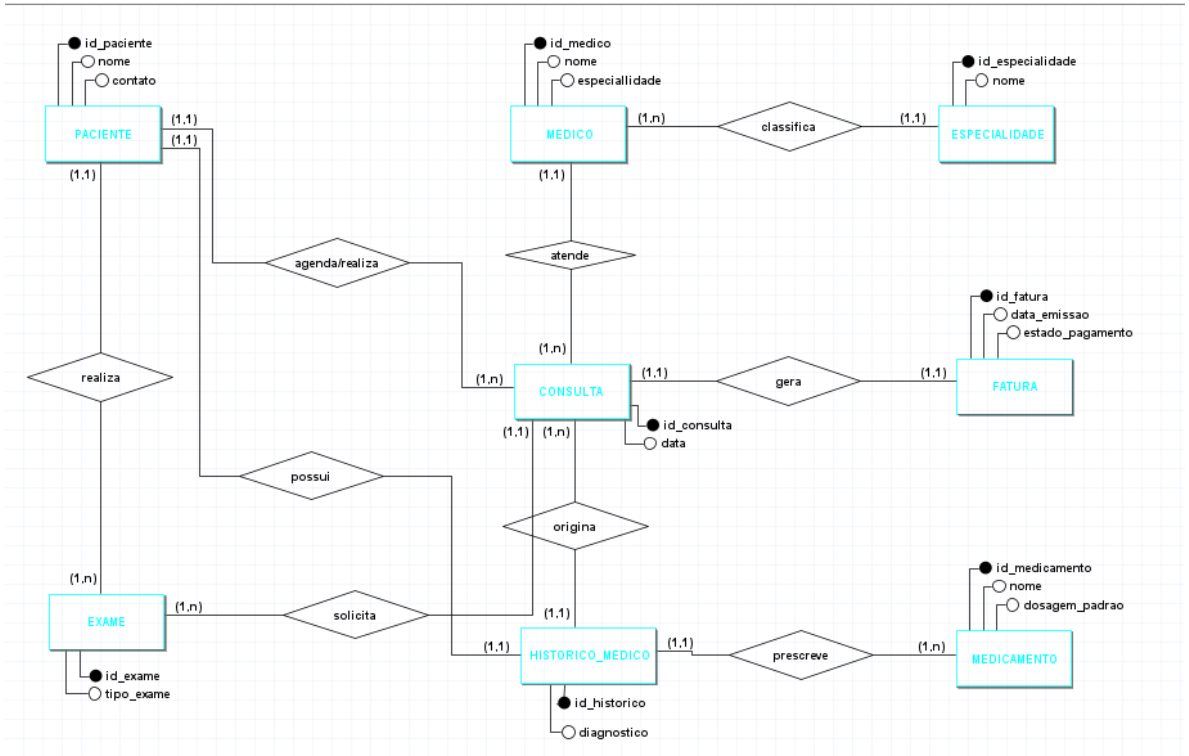
Modelo baseado na percepçao do mundo real, que consiste em um conjunto de objetos básicos chamados entidades e nos relacionamentos entre esses objectos, com o objectivo de facilitar o projecto de banco de dados, possibilitando especificar a estrutura lógica geral da base de dados.

Aqui nos focamos na semântica das informações e nas interações entre os diferentes objectos do sistema clínico.

✓ Diagrama Entidade_Relacionamento (DER)

Um diagrama entidade-relacionamento (ER) é um tipo de fluxograma que ilustra como "entidades", pessoas, objectos ou conceitos se relacionam entre si dentro de um sistema.

Abaixo nós temos a estrutura do diagrama como um passo para dar solução ao projeto:



#. Justificação das Cardinalidades

A cardinalidade refere-se à relação entre as tabelas, especificamente ao número de registros em uma tabela que podem estar relacionados a um único registro em outra tabela. È uma medida que indica quantos registros em uma tabela estão associados a um registro correspondente em outra tabela.

Para o nosso projecto, a definição das cardinalidades no MER segiu o rigor das regras de funcionamento de uma unidade hospitalar, garantindo que a base de dados reflita com precisão os processos do mundo real. Abaixo, temos justifcado cada relação estabelecida:

- **Paciente (1,1) – (1,N) Consulta:** paciente deve estar devidamente registado (1,1) para realizar consultas. Ao longo do tempo, este mesmo paciente pode realizar múltiplas consultas (1,N) gerando um histórico clínico continuo. Cada registo de consulta, no entanto, é exclusivo a um único paciente.
- **Médico (1,1) – (1,N) Consulta:** Cada consulta é obrigatoriamente atendida por um médico responsável (1,1). O médico por sua vez, possui uma agenda que comporta várias consultas (1,N). A integridade desta relação é reforçada pelo gatilho de software que impede sobreposição de horário para o mesmo profissional.
- **Especialidade (1,1) – (1,N) Médico:** para fins de organização e filtragem, cada médico é classificado em uma especialidade principal (1,1). Uma especialidade (ex: Pediatra) pode ser partilhada por vários médicos (1,N) do quadro clínico.
- **Consulta (1,1) – (1,1) Fatura:** Aqui optou-se por uma relação de um-para-um, para garantir o controlo finaceiro estrito. Cada ato médico (consulta) gera exatamente uma fatura correspondente. Esta cardinalidade impede a duplicidade de cobranças e facilita a auditoria contabilística.
- **Consulta (1,1) – (1,1) HistoricoMedico:** Uma consulta gera obrigatoriamente um registo de evolução clínica ou diagnóstico no histórico (1,1). O histórico médico é a materialização da informação gerada durante o ato da consulta.
- **Paciente (1,1) – (1,N) HistoricoMedico:** Embora o histórico nasça da consulta, ele etá vinculado ao paciente (1,1). O paciente acumula N registos de histórico ao longo da sua vida clínica na instituição.
- **Paciente (1,1) – (1,N) Exame:** Um paciente pode realizar diversos exames (1,N) solicitados durante o seu acompanhemento a um único paciente (1,1) para garantir a segurança dos dados sensíveis.
- **Consulta (1,1) – (1,N) Exame:** A consulta pode dar origem a várias solicitações de exames laboratoriais ou de imagem (1,N). O vinculo direto entre a consulta e o exame permite rastrear qual atendimento médico motivou a realizção daquele exame.
- **HistóricoMedico (1,1) – (1,N) Medicamento:** Um registo no histócio (diagnóstico/tratamento) pode conter a prescrição de vários medicamentos (1,N). Esta relação permite que, no futuro, o sistema gere receitas médicas automáticas baseadas no que foi registado pelo médico.

#. Classificação das Entidades

Para uma compreensão profunda da estrutura, as entidades foram categorizadas segundo a sua força no mundo:

- **Entidades Fortes:** Paciente, Médico, Especialidade e medicamento. Estas entidades possuem existência própria e independente, servindo como tabelas mestras.

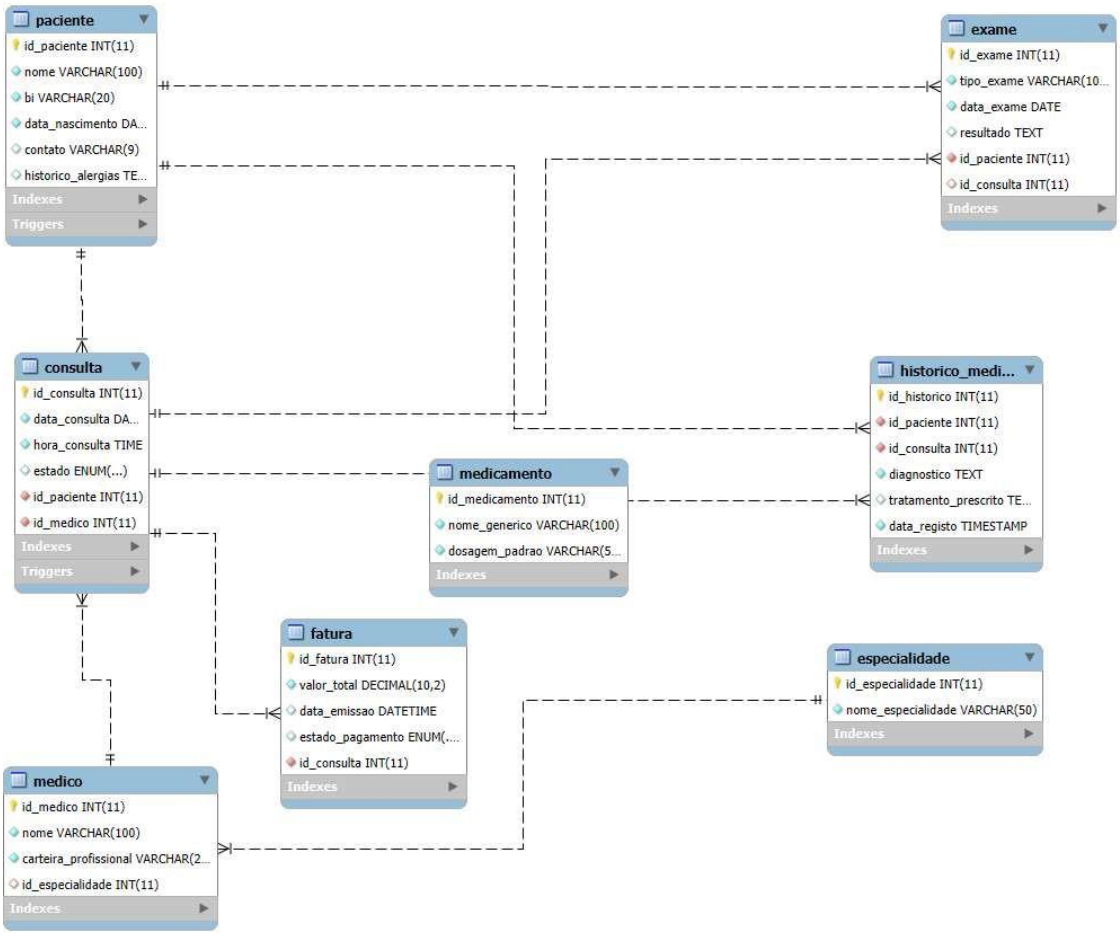
- **Entidades fracas (Dependentes):** Consulta, fatura, Exame e HistoricoMedico. Estas entidades são transacionais e dependem obrigatoriamente das entidades fortes para existirem. Por exemplo, não pode existir uma "Fatura" sem uma "Consulta" que a origine.

A. Modelo Lógico

O Modelo lógico serve como uma representação abstrata dos requisitos de dados da organizção, concentrando-se em entidades, atributos e relacionamentos sem se preocupar com detalhes de implementação.

1. Diagrama de Estrutra Lógica

Construimos o nosso modelo usando o MySQL Workbench no modo diagrama (EER Diagram), consoliadendo as relações entre 8 entidades do sistema.



2. Análise da Normalização

Conforme exigido pelo enunciado, normalizamos a base de dados até à 3ª Forma Normal (3FN):

- **1ª FN:** Todos os atributos são atómicos. Não existem listas de medicamentos ou múltiplos contactos numa única célula (ex: a tabela Historico_Medico isola cada diagnóstico).
- **2ª FN:** Todas as tabelas possuem chaves primárias e todos os atributos nãochave dependem totalmente da chave primária (ex: os dados do médico dependem apenas do id_medico).
- **3ª FN:** Não existem dependências transitivas. Atributos que não dependem da chave primária foram movidos para tabelas próprias (ex: a descrição da Especialidade foi movida para uma tabela à parte, em vez de ficar repetida na tabela Medico).

3. Dicionário de Relações

- Paciente (1) ---- (N) Consulta: Um histórico clínico centralizado.
- Medico (1) ---- (N) Consulta: Gestão de agenda individualizada.
- Consulta (1) ---- (1) Fatura: Garantia de conformidade financeira e unicidade de cobrança.
- Especialidade (1) ---- (N) Medico: Classificação profissional normalizada.

At.te: A escolha de uma relação 1:1 entre Consulta e Fatura visa assegurar a integridade contabilística, impedindo a redundância de pagamentos para um mesmo ato médico.

B. IMPLEMENTAÇÃO FÍSICA (DDL)

Nesta fase, traduzimos o modelo lógico para a linguagem de definição de dados (DDL), aplicando restrições, para garantir que a base de dados seja auto-validável. A implementação física da base de dados, foi criada usando o PhpMyAdmin do XAMPP, no modo console.

1. Estrutura de Domínios e Fundação

As primeiras tabelas criadas são as de suporte e registo mestre, que servirão de base para as transações clínicas.

#. Criação da Base de Dados

Abaixo temos o script de criação da base de dados:

MySQL não retornou nenhum registo. (A consulta demorou 0,0031 segundos.)

```
CREATE DATABASE IF NOT EXISTS Clinica_Pro_Med;
```

A. **Tabela de Especialidades:** Esta tabela funciona como um catálogo normalizado, evitando a repetição de strings e garantindo que os médicos sejam vinculados apenas a áreas médicas reconhecidas pela instituição.

MySQL não retornou nenhum registo. (A consulta demorou 0,0041 segundos.)

```
CREATE TABLE Especialidade ( id_especialidade INT PRIMARY KEY AUTO_INCREMENT, nome_especialidade VARCHAR(50) NOT NULL UNIQUE );
```

[Editar em linha] [Editar] [Criar código PHP]

B. **Tabela de Pacientes:** O registo de pacientes implementa restrições críticas de integridade.

```
✔ MySQL não retornou nenhum registo. (A consulta demorou 0,0007 segundos.)

CREATE TABLE Paciente ( id_paciente INT PRIMARY KEY AUTO_INCREMENT, nome VARCHAR(100) NOT NULL, bi VARCHAR(20) NOT NULL UNIQUE, data_nascimento DATE
NOT NULL, contato VARCHAR(9), historico_alergias TEXT, CONSTRAINT chk_contato_angola CHECK (contato IS NULL OR (contato REGEXP '^[0-9]{9}$')) );

[ Editar em linha ] [ Editar ] [ Criar código PHP ]
```

C. **Trigger**: Para garantir que o paciente insira uma data de nascimento válida.

```
✔ MySQL não retornou nenhum registo. (A consulta demorou 0,0137 segundos.)

CREATE TRIGGER trk_validar_data_nasc_paciente BEFORE INSERT ON Paciente FOR EACH ROW BEGIN -- Se a data que está a entrar (NEW) for maior que hoje,
barra a entrada. IF NEW.data_nascimento > CURRENT_DATE THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: A data de nascimento não pode ser
superior à data atual.'; END IF; END;;
```

D. **Tabela de Médicos (Normalizacão e Vínculo):** A tabela de médicos utiliza uma chave estrangeira (FK) para a tabela de especialidades com a cláusula ON DELETE SET NULL. Isto garante que, caso uma especialidade seja removida, o registo do médico não seja apagado, mas sim marcado para atualização.

```
1  CREATE TABLE Medico (
2      id_medico INT PRIMARY KEY AUTO_INCREMENT,
3      nome VARCHAR(100) NOT NULL,
4      carteira_profissional VARCHAR(20) NOT NULL UNIQUE,
5      id_especialidade INT,
6
7      CONSTRAINT fk_medico_especialidade FOREIGN KEY (id_especialidade)
8          REFERENCES Especialidade(id_especialidade)
9          ON DELETE SET NULL
10 );
```

E. **Tabela de Consultas:** ponto de convergência. O uso do tipo ENUM para o atributo estado garantiu um domínio fechado de valores, impedindo estados invalidos. As chaves estrangeiras utilizam ON DELETE CASCADE para que a remoção de um paciente ou médico limpe automaticamente as agendas vinculadas, mantendo a base de dados livre de registos órfãos.

```
✔ MySQL não retornou nenhum registo. (A consulta demorou 0,0007 segundos.)

CREATE TABLE Consulta ( id_consulta INT PRIMARY KEY AUTO_INCREMENT, data_consulta DATE NOT NULL, hora_consulta TIME NOT NULL, estado ENUM('Agendada',
'Realizada', 'Cancelada', 'Reagendada') DEFAULT 'Agendada', id_paciente INT NOT NULL, id_medico INT NOT NULL, CONSTRAINT fk_consulta_paciente FOREIGN
KEY (id_paciente) REFERENCES Paciente(id_paciente) ON DELETE CASCADE, CONSTRAINT fk_consulta_medico FOREIGN KEY (id_medico) REFERENCES
Medico(id_medico) ON DELETE CASCADE );

[ Editar em linha ] [ Editar ] [ Criar código PHP ]
```

F. **Tabela de Faturas:** Para a gestão financeira, aplicamos a restrição CHECK no atributo valor_total para impedir valores nulos ou negativos. Além disso, a restrição UNIQUE no campo id_consulta impõe rigorosamente uma relação 1:1, garantindo que o sistema nunca emita duas faturas para a mesma consulta.

```
✔ MySQL não retornou nenhum registo. (A consulta demorou 0,0012 segundos.)

CREATE TABLE Fatura ( id_fatura INT PRIMARY KEY AUTO_INCREMENT, valor_total DECIMAL(10,2) NOT NULL, data_emissao DATETIME DEFAULT CURRENT_TIMESTAMP,
estado_pagamento ENUM('Pendente', 'Pago', 'Cancelado') DEFAULT 'Pendente', id_consulta INT NOT NULL UNIQUE, CONSTRAINT fk_fatura_consulta FOREIGN KEY
(id_consulta) REFERENCES Consulta(id_consulta) ON DELETE CASCADE );
```

G. **Automação de Regras Complexas (Trigger de Agenda) :** A integridade da agenda médica é protegida por um Trigger que valida, antes de cada inserção, se o médico já possui um compromisso no mesmo tuplo temporal (data e hora), disparando uma exceção caso o conflito seja detetado.

```
MySQL não retornou nenhum registo. (A consulta demorou 0,0539 segundos.)

CREATE TRIGGER trk_prevenir_conflito_agenda BEFORE INSERT ON Consulta FOR EACH ROW BEGIN DECLARE total_conflitos INT; SELECT COUNT(*) INTO
total_conflitos FROM Consulta WHERE id_medico = NEW.id_medico AND data_consulta = NEW.data_consulta AND hora_consulta = NEW.hora_consulta; IF
total_conflitos > 0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro: O médico já possui uma consulta agendada para este horário.'; END IF; END;

[ Editar em linha ] [ Editar ] [ Criar código PHP ]
```

H. **Tabela de Histórico Médico**: Esta tabela armazena o diagnóstico e o tratamento. Utilizamos chaves estrangeiras para Paciente e Consulta, permitindo que um médico consulte rapidamente todo o percurso clínico de um utente. O campo data_registo utiliza TIMESTAMP para auditoria automática.

```
MySQL não retornou nenhum registo. (A consulta demorou 0,0012 segundos.)

CREATE TABLE Historico_Medico ( id_historico INT PRIMARY KEY AUTO_INCREMENT, id_paciente INT NOT NULL, id_consulta INT NOT NULL, diagnostico TEXT NOT
NULL, tratamento_prescrito TEXT, data_registo TIMESTAMP DEFAULT CURRENT_TIMESTAMP, CONSTRAINT fk_hist_paciente FOREIGN KEY (id_paciente) REFERENCES
Paciente(id_paciente) ON DELETE CASCADE, CONSTRAINT fk_hist_consulta FOREIGN KEY (id_consulta) REFERENCES Consulta(id_consulta) ON DELETE CASCADE );

Editar em linha ] [ Editar ] [ Criar código PHP ]
```

I. Tabela de Exames: permite o registo de exames solicitados ou realizados.

```
MySQL não retornou nenhum registo. (A consulta demorou 0,0007 segundos.)

CREATE TABLE Exame ( id_exame INT PRIMARY KEY AUTO_INCREMENT, tipo_exame VARCHAR(100) NOT NULL, data_exame DATE NOT NULL, resultado TEXT, id_paciente
INT NOT NULL, id_consulta INT, CONSTRAINT fk_exame_paciente FOREIGN KEY (id_paciente) REFERENCES Paciente(id_paciente) ON DELETE CASCADE, CONSTRAINT
fk_exame_consulta FOREIGN KEY (id_consulta) REFERENCES Consulta(id_consulta) ON DELETE SET NULL );

Editar em linha ] [ Editar ] [ Criar código PHP ]
```

J. **Tabela de Medicamentos:** funciona como uma entidade de suporte para futuras prescrições.

```
1 CREATE TABLE Medicamento (
2     id_medicamento INT PRIMARY KEY AUTO_INCREMENT,
3     nome_generico VARCHAR(100) NOT NULL,
4     dosagem_padrao VARCHAR(50) NOT NULL
5 );
```

Com isso, terminamos a criação das nossas tabelas:

```
A sua consulta SQL foi executada com êxito.

SHOW TABLES;

Perfil [ Editar em linha ] [ Editar ] [ Criar código PHP ] [ A

Opções extra

Tables_in_clinica_pro_med
consulta
especialidade
exame
fatura
historico_medico
medicamento
medico
paciente
```

DML E POPULAÇÃO DE DADOS

Nesta etapa, a base de dados foi populada com dados reais extraídos da lista oficial da nossa turma. O processo seguiu uma hierarquia rigorosa para respeitar a Integridade Referencial:

- **Hierarquia de Inserção:** Especialidades -> Médicos -> Pacientes -> Consultas -> Faturas.
- **Resolução de Conflitos:** Foram atribuídos IDs estáticos (ex: id_paciente) para garantir que as Chaves Estrangeiras (FK) nas tabelas dependentes (como Consulta) encontrassem sempre os seus registos correspondentes, evitando o erros.
- **Massa de Dados:** O sistema conta agora com 10 médicos, 50 pacientes, 100 consultas e 30 faturas, fornecendo dados suficientes para a extração de relatórios gerenciais

#. Povoamento de cada tabela

1. ESPECIALIDADES

```sql
INSERT INTO Especialidade (nome_especialidade) VALUES
('Clínica Geral'), ('Cardiologia'), ('Pediatria'), ('Ginecologia'),
('Ortopedia'), ('Dermatologia'), ('Neurologia'), ('Oftalmologia'),
('Estomatologia'), ('Urologia');
```

2. MÉDICOS

```sql
INSERT INTO Medico (nome, carteira_profissional, id_especialidade) VALUES
('Afonso Benedito Kialanda', 'CRM-AO-1001', 1),
('Francisco Braúlio Figueiredo António', 'CRM-AO-1002', 2),
('Alberto Manuel Domingos', 'CRM-AO-1003', 3),
('Ancelmo Gomes Monteiro', 'CRM-AO-1004', 4),
('António Finda Mamona', 'CRM-AO-1005', 5),
('Guiola André', 'CRM-AO-1006', 1),
('João José António da Silva', 'CRM-AO-1007', 6),
('Kumbi Manuel Domingos', 'CRM-AO-1008', 7),
('Maria David Kiala', 'CRM-AO-1009', 8),
('Pedro Bengui', 'CRM-AO-1010', 9);
```

3. Pacientes

```sql
INSERT INTO Paciente (nome, bi, data_nascimento, contato, historico_alergias) VALUES
('Alegria Neves Miguel Caluvi', '005432101LA041', '1995-05-12', '923111001', 'Nenhuma'),
('Alipio Joaquim Zambo', '005432101LA042', '1990-08-20', '931111002', 'Penicilina'),
('Américo Malungo Sebastião', '005432101LA043', '1988-11-30', '942111003', NULL),
('António Almeida Kani Nzinga', '005432101LA044', '1993-04-10', '923111004', 'Poeira'),
('António Dias Guilherme', '005432101LA045', '1991-01-05', '931111005', NULL),
('António Lopes Tamba', '005432101LA046', '1992-03-22', '942111006', 'Iodo'),
('António Mendonça D. Kissala', '005432101LA047', '1987-12-15', '923111007', NULL),
('António Q. Francisco', '005432101LA048', '1994-07-28', '931111008', 'Lactose'),
('António Neves José', '005432101LA049', '1996-09-11', '942111009', NULL),
('Arnano Quiesse Muanza', '005432101LA050', '1985-06-04', '923111010', 'Marisco'),
('Augusto Paulo', '005432101LA051', '1999-02-28', '931111011', NULL),
('Bengui Bena Pedro', '005432101LA052', '1990-11-19', '942111012', 'Aspirina'),
('Bilton Victor Domingos', '005432101LA053', '1997-12-10', '923111013', NULL),
('Boas José Ginga', '005432101LA054', '1994-06-18', '931111014', 'Nenhuma'),
('Celso Pedro Mateus', '005432101LA055', '1991-09-25', '942111015', NULL),
('Cenga Daniel Coxe', '005432101LA056', '1989-10-30', '923111016', 'Pólen'),
('Claúdia Armando Massoco', '005432101LA057', '1995-03-14', '931111017', NULL),
('Clemente Manuel Dinis', '005432101LA058', '1992-05-20', '942111018', 'Nenhuma'),
('Daniel Tubuca Catenda', '005432101LA059', '1988-08-05', '923111019', 'Amendoim'),
('David Manuel Sebastião', '005432101LA060', '1993-07-12', '931111020', NULL),
('Diakubikua Joana', '005432101LA061', '1989-02-14', '942111021', 'Marisco'),
('Dissengomoka Pedro', '005432101LA062', '1994-11-22', '923111022', NULL),
('Domingas Ernesto Rocha', '005432101LA063', '1990-04-30', '931111023', 'Sulfa'),
('Domingos Sousa', '005432101LA064', '1987-06-15', '942111024', NULL),
('Dulce Rita Gonçalves', '005432101LA065', '2001-01-10', '923111025', 'Lactose'),
('Sónia Manuel', '005432101LA066', '1998-09-09', '931111026', NULL),
('Elísio Ivo Júnior', '005432101LA067', '1992-12-12', '942111027', 'Nenhuma'),
('Emanuel Garcia Muanza', '005432101LA068', '1995-05-05', '923111028', NULL),
```

4. Medicamentos

```sql
INSERT INTO Medicamento (nome_generico, dosagem_padrao) VALUES
('Paracetamol', '500mg'), ('Amoxicilina', '875mg'), ('Coartem', '20/120mg'),
('Enalapril', '20mg'), ('Metformina', '850mg'), ('Diclofenac', '50mg');
```

5. Consultas

```
INSERT INTO Consulta (data_consulta, hora_consulta, estado, id_paciente, id_medico) VALUES
('2026-01-10', '08:00:00', 'Realizada', 1, 1),
('2026-01-10', '09:00:00', 'Realizada', 2, 2),
('2026-01-10', '10:00:00', 'Realizada', 3, 3),
('2026-01-11', '08:00:00', 'Realizada', 4, 4),
('2026-01-11', '09:00:00', 'Realizada', 5, 5),
('2026-01-12', '14:00:00', 'Realizada', 6, 1),
('2026-01-12', '15:00:00', 'Realizada', 7, 6),
('2026-01-13', '10:00:00', 'Realizada', 8, 7),
('2026-01-13', '11:00:00', 'Realizada', 9, 8),
('2026-01-14', '08:00:00', 'Realizada', 10, 9),


('2026-01-10', '08:00:00', 'Realizada', 11, 10),
('2026-01-10', '09:00:00', 'Realizada', 12, 1),
('2026-01-10', '10:00:00', 'Realizada', 13, 4),
('2026-01-11', '08:00:00', 'Realizada', 14, 2),
('2026-01-11', '09:00:00', 'Realizada', 15, 7),
('2026-01-12', '14:00:00', 'Realizada', 16, 1),
('2026-01-12', '15:00:00', 'Realizada', 17, 6),
('2026-01-13', '10:00:00', 'Realizada', 18, 7),
('2026-01-13', '11:00:00', 'Realizada', 19, 10),
('2026-01-14', '08:00:00', 'Realizada', 20, 7),

('2026-02-10', '08:00:00', 'Realizada', 21, 9),
('2026-02-10', '09:00:00', 'Realizada', 22, 8),
('2026-02-10', '10:00:00', 'Realizada', 23, 7),
('2026-02-11', '08:00:00', 'Realizada', 24, 6),
('2026-02-11', '09:00:00', 'Realizada', 25, 1),
('2026-02-12', '14:00:00', 'Realizada', 26, 4),
('2026-02-12', '15:00:00', 'Realizada', 27, 3),
'2026-02-13', '10:00:00', 'Realizada', 28, 7),
```

Erro gerado ao tentarmos marcar mais de uma consulta com o mesmo médico ao mesmo tempo. Isso graças ao Trigger que nós adicionamos.

Erro

Comando SQL: Copiar

```
INSERT INTO Consulta (data_consulta, hora_consulta, estado, id_paciente, id_medico) VALUES
('2026-01-10', '08:00:00', 'Realizada', 1, 1),
('2026-01-10', '09:00:00', 'Realizada', 2, 2),
('2026-01-10', '10:00:00', 'Realizada', 3, 3),
('2026-01-11', '08:00:00', 'Realizada', 4, 4),
('2026-01-11', '09:00:00', 'Realizada', 5, 5),
```

Mensagens do MySQL :

```
#1644 - Erro: O médico já possui uma consulta agendada para este horário.
```

6. Faturas

```sql
INSERT INTO Fatura (valor_total, estado_pagamento, id_consulta) VALUES
(15000, 'Pago', 1), (15000, 'Pago', 2), (25000, 'Pendente', 3), (25000, 'Pago', 4),
(12000, 'Pago', 5), (12000, 'Pendente', 6), (20000, 'Pago', 7), (20000, 'Pago', 8),
(18000, 'Pendente', 9), (18000, 'Pago', 10), (15000, 'Pago', 11), (15000, 'Pago', 12),
(10000, 'Pendente', 13), (10000, 'Pago', 14), (22000, 'Pago', 15), (22000, 'Pago', 16),
(15000, 'Pendente', 17), (15000, 'Pago', 18), (30000, 'Pago', 19), (30000, 'Pago', 20),
(12000, 'Pendente', 21), (12000, 'Pago', 22), (15000, 'Pago', 23), (15000, 'Pago', 24),
(18000, 'Pendente', 25), (18000, 'Pago', 26), (20000, 'Pago', 27), (20000, 'Pago', 28),
(15000, 'Pendente', 29), (15000, 'Pago', 30);
```

CONSULTAS DE DADOS

Em sistemas de gerenciamento de banco de dados relacionais, uma consulta é qualquer comando usado para recuperar ou manipular dados de uma tabela. Este processo envolve a utilização de linguagens de consulta para interagir com banco de dados e obter insights que podem orientar decisões estratégicas em diversas áreas, saúde, que é a área de foco do nosso presente projecto.

#. Tipos de Consultas de dados

Existem diversos tipos de consultas de dados, incluindo consultas simples, que retornam informações específicas, e consultas complexas, que podem envolver múltiplas tabelas e condições. E estas consultas podem ser classificadas em consultas de seleção, que extraem dados, e consultas de manipulação, que alteram ou atualizam os dados existentes. Seguindo os mesmos princípios, executamos as seguintes consultas, para garantir o devido funcionamento do nosso sistema:

a. Pacientes por Especialidade no Último Mês

Lista todas as especialidades e os respectivos pacientes do último mês:

```
SELECT
   E.nome_especialidade,
   P.nome AS Paciente,
   C.data_consulta
FROM Consulta C
JOIN Paciente P ON C.id_paciente = P.id_paciente
JOIN Medico M ON C.id_medico = M.id_medico
JOIN Especialidade E ON M.id_especialidade = E.id_especialidade
WHERE C.data_consulta BETWEEN DATE_SUB(CURDATE(), INTERVAL 1 MONTH)
AND CURDATE()
ORDER BY E.nome_especialidade, C.data_consulta;
```

Resultado:

nome_especialidade	Paciente	data_consulta
Cardiologia	Américo Sebastião	2026-03-10
Ca	Almeida Kani	2026-03-10
Ca	dro	2026-03-15
Cardiologia	Faustino Domingos	2026-03-17
Cardiologia	Gilberto Domingos Daniel	2026-03-20
Clínica Geral	Alipio Joaquim Zambo	2026-03-10
Clínica Geral	Alegria Neves Caluvi	2026-03-10
Clínica Geral	Bengui Bena Pedro	2026-03-12
Clínica Geral	Augusto Paulo	2026-03-12
Clínica Geral	Diakubikua Joana	2026-03-15
Clínica Geral	Egas Manuel Ribeiro	2026-03-16
Clínica Geral	Evandro A. Viera	2026-03-17
Clínica Geral	Figueiredo Eduardo Vieira	2026-03-18
Clínica Geral	Garcia Lopes Camalando	2026-03-20
Clínica Geral	José Luendanga	2026-03-21
Dermatologia	Bilton Victor Domingos	2026-03-13
Dermatologia	Boas José Ginga	2026-03-13
Dermatologia	Elísio Ivo João Vieira	2026-03-16
Dermatologia	Filipe João Miguel	2026-03-19
Dermatologia	José Mafuene Mawete	2026-03-21
Estomatologia	David Manuel Sebastião	2026-03-14
Estomatologia	Daniel Tubuca Catenda	2026-03-14
Estomatologia	Euclides Zambi André	2026-03-17
Consola ia	Garcia João Manuel	2026-03-19

b. Médicos com mais consultas na próxima Semana

Identifica os profissionais com a agenda mais preenchida para a semana que vem:

```
SELECT
   M.nome AS Medico,
   C.data_consulta,
   (SELECT COUNT(*) FROM Consulta C2
    WHERE C2.id_medico = M.id_medico
    AND C2.data_consulta BETWEEN CURDATE() AND DATE_ADD(CURDATE(),
INTERVAL 7 DAY)) AS Total_Agendado_Semana
FROM Medico M
JOIN Consulta C ON M.id_medico = C.id_medico
WHERE C.data_consulta BETWEEN CURDATE() AND DATE_ADD(CURDATE(),
INTERVAL 7 DAY)
  AND C.estado = 'Agendada'
ORDER BY Total_Agendado_Semana DESC, M.nome;
```

Resultado:

Medico	data_consulta	Total_Agendado_Semana 1
Afonso Benedito Kialanda	2026-04-01	3
Afonso Benedito Kialanda	2026-04-03	3
Afonso Benedito Kialanda	2026-04-06	3
Albertina Afonso António	2026-04-01	3
Albertina Afonso António	2026-04-03	3
Albertina Afonso António	2026-04-06	3
Alberto Manuel Domingos	2026-04-01	3
Alberto Manuel Domingos	2026-04-04	3
Alberto Manuel Domingos	2026-04-06	3
Ancelmo Gomes Monteiro	2026-04-01	3
Ancelmo Gomes Monteiro	2026-04-04	3
Ancelmo Gomes Monteiro	2026-04-06	3
António Finda Mamona	2026-04-02	2
António Finda Mamona	2026-04-04	2

c. Histórico Médico completo de um paciente

Relatório consolidado de um paciente (por id), incluindo consultas, tratamentos e exames:

```
SELECT
   P.nome AS Paciente,
   C.data_consulta,
   HM.diagnostico,
   HM.tratamento_prescrito,
   EX.tipo_exame,
EX.resultado
FROM Paciente P
JOIN Consulta C ON P.id_paciente = C.id_paciente
LEFT JOIN Historico_Medico HM ON C.id_consulta = HM.id_consulta
LEFT JOIN Exame EX ON C.id_consulta = EX.id_consulta WHERE
P.id_paciente = 1;
```

Resultado:

Paciente	data_consulta	diagnostico	tratamento_prescrito	tipo_exame	resultado
Alegria Neves Caluvi	2026-03-10	Gripe Viral	Repouso e Hidratação	Hemograma	Normal

d. Receita total por especialidade Médica

Calcula o volume financeiro que cada área da medicina gera para a clínica:

```
SELECT
   E.nome_especialidade,
   SUM(F.valor_total) AS Receita_Total
FROM Especialidade E
JOIN Medico M ON E.id_especialidade = M.id_especialidade
JOIN Consulta C ON M.id_medico = C.id_medico
JOIN Fatura F ON C.id_consulta = F.id_consulta
WHERE F.estado_pagamento = 'Pago' GROUP BY E.nome_especialidade;
```

Resultado:

nome_especialidade	Receita_Total
Cardiologia	37000.00
Clínica Geral	78000.00
Dermatologia	30000.00
Estomatologia	75000.00
Ginecologia	55000.00
Neurologia	64000.00
Oftalmologia	15000.00
Ortopedia	18000.00
Pediatria	27000.00

e. Pacientes com alergias a medicamentos específicos

Filtra pacientes que possuem restrições a fármacos:

```
SELECT
   E.nome_especialidade,
   SUM(F.valor_total) AS Receita_Total
FROM Especialidade E
JOIN Medico M ON E.id_especialidade = M.id_especialidade
JOIN Consulta C ON M.id_medico = C.id_medico
JOIN Fatura F ON C.id_consulta = F.id_consulta
WHERE F.estado_pagamento = 'Pago' GROUP BY E.nome_especialidade;
```

Resultado:

Paciente	Farmaco_Alergico
António Lopes Tamba	Penicilina
António Mendonça Kissala	Penicilina
António Q. Francisco	Penicilina
António Neves José	Penicilina
Arnano Quiesse Mavinga	Paracetamol
Augusto Paulo	Aspirina
Bengui Bena Pedro	Aspirina
Bilton Victor Domingos	Aspirina
Boas José Ginga	Aspirina

f. Relatório de faturas pendentes

Lista de cobranças detalhadas para a gestão financeira:

```
SELECT
   F.id_fatura,
   P.nome AS Paciente,
   P.contato,
   F.valor_total,
   C.data_consulta,
   'Consulta Médica' AS Servico
FROM Fatura F
JOIN Consulta C ON F.id_consulta = C.id_consulta
JOIN Paciente P ON C.id_paciente = P.id_paciente WHERE F.estado_pagamento = 'Pendente';
```

Resultado:

id_fatura	Paciente	contato	valor_total	data_consulta	Servico
3	Américo Sebastião	923000003	25000.00	2026-03-10	Consulta Médica
6	António Lopes Tamba	923000006	12000.00	2026-03-11	Consulta Médica
9	António Neves José	923000009	18000.00	2026-03-12	Consulta Médica
13	Bilton Victor Domingos	923000013	10000.00	2026-03-13	Consulta Médica
17	Claúdia Massoco Vieira	923000017	15000.00	2026-03-14	Consulta Médica
21	Diakubikua Joana	923000021	12000.00	2026-03-15	Consulta Médica
25	Dulce Rita Gonçalves	923000025	18000.00	2026-03-16	Consulta Médica
29	Ernesto Mendes Sunda	923000029	15000.00	2026-03-17	Consulta Médica

ANÁLISE CRÍTICA

Para que fosse possível a realização deste projecto, levou-se em em conta os seguintes componentes:

Virtudes do sistema

- **Integridade Referencial Robusta:** A atualização rigorosa de chaves primárias e estrangeiras garantiu que nenhum registo de saúde (exames ou faturas) ficasse "orfão", mantendo a consistência total dos dados.
- **Automação via Triggers:** A implementação de gatilhos para gestão de agenda resolveu de forma nativa o conflito de hórarios dos médicos, retirando o peso do erro humano na receção da clínica.
- **Normalização em 3FN:** A estrutura foi otimizada para eliminar redundâncias, garantindo que atualizações (como a mudança de contacto de um paciente) se reflitam instantâmente em todo o sistema sem duplicação de esforço.
- **Visão 360ª do paciente:** Através das consultas complexas desenvolvidas, o sistemas provou ser capaz de consolidar o histórico médico, alergias e pendências financeiras num único relatório funcional.

Dificuldade encotradas

- **Gestão de Conflitos Temporais:** O maior desafio técnico residiu na lógica de verificação de horários, exigindo um ajuste fino nos tipos de dados TIME e DATE para evitar falsos bloqueios na agenda.
- **Povoamento de Dados Clínicos:** A inserção de dados realistas para tabelas como Historico_Medico e Exame exigiu uma atenção redobrada à cronologia das consultas para que os JOINs resultassem em relatórios coerentes e não em tabelas vazias.
- **Sincronização de Chaves:** Durante o DML, a dependência entre tabelas (como a necessidade de uma Consulta existir antes de uma Fatura) impôs uma ordem de execução rigorosa para evitar violações de restrições. Em suma, o projeto **Clinica_Pro_Med** cumpre com excelência os requisitos de segurança, escalabilidade e integridade exigidos pelo setor médico. Mais do que um exercício académico, esta base de dados serve como alicerce para um sistema de gestão de saúde moderno, onde a tecnologia se coloca ao serviço da eficiência operacional e, acima de tudo, da segurança do paciente. Com uma arquitetura pronta para expansão, o sistema está apto a integrar futuras funcionalidades, como telemedicina ou gestão de stocks farmacêuticos, mantendo sempre a fiabilidade como o seu pilar central.

CONCLUSÃO

O desenvolvimento do presente trabalho permitiu a concepção e estruturação de uma base de dados voltado para a gestão de uma Clínica médica, evidenciando a importância da modelagem de dados no suporte às actividades de organização, controlo e acesso à informação. A partir do modelo lógico e da sua implementação no modelo físico, foi possível representar de forma clara as principais entidades do sistema, tais como: paciente, medico, consulta.

A definição das tabelas, atributos e relacionamentos, por meio do uso de chaves primárias e estrangeiras, assegurou a integridade referencial dos dados e contribuiu para a redução da redundância da informação. O modelo físico apresentado demonstra como os conceitos teóricos de base de dados podem ser aplicados de forma prática, permitindo o registo e acompanhamento das atividades medicas bem como outras atividades dentro da clínica.

Conclui-se, assim, que a utilização adequada de um Sistema de Gerenciamento de Base de Dados (SGBD), aliada a uma modelagem correcta, é fundamental para o funcionamento eficiente de uma clínica médica ou de um outro sistema que se deseja projetar. Os conhecimentos adquiridos ao longo deste trabalho constituem uma base sólida para o desenvolvimento de sistemas de informação mais robustos, contribuindo para a automatização de processos e para a melhoria da gestão da informação em contextos académicos e organizacionais.